

Hoja de Referencia MIPS32

Arquitectura de Computadores

Estudiante: David Serrano

14 de mayo de 2026

Instrucciones de Aritmética y Lógica

Instrucción	Sintaxis	Tipo	Descripción
add	add \$d, \$s, \$t	R	Suma: $d = s + t$. Reporta overflow.
addi	addi \$t, \$s, imm	I	Suma inmediata: $t = s + constante$.
sll	sll \$d, \$t, shamt	R	Desplazamiento lógico a la izquierda (Shift).

1. **ADD**

En la arquitectura **MIPS32**, la instrucción **add** es una de las operaciones fundamentales de aritmética. Se utiliza para sumar los valores contenidos en dos registros y almacenar el resultado en un tercer registro.

A continuación, te presento el desglose técnico de esta instrucción siguiendo el formato de algoritmo y las especificaciones de arquitectura.

Tipo de Instrucción: Tipo-R (Registro).

Se utiliza para sumar los valores contenidos en dos registros y almacenar el resultado en un tercer registro.

Características de la instrucción add: Tipo de Instrucción: Tipo-R (Registro).

Operación: Realiza una suma aritmética de enteros con detección de desbordamiento (overflow). Si el resultado de la suma excede la capacidad del registro, el procesador genera una excepción.

Sintaxis: add rd,rs,rt

rd: Registro destino (donde se guarda el resultado).

rs: Primer registro fuente.

rt: Segundo registro fuente.

EJEMPLO:

”Supongamos: $s1 = 10$, $s2 = 20$

add s0, s1, s2

seria: $s0 = s1 + s2$ ”

2. ADDI

En la arquitectura **MIPS32** la instrucción **addi** (Add Immediate) se utiliza para sumar el valor de un registro con un valor constante (un número entero inmediato) que viene incluido directamente en la propia instrucción.

A diferencia de **add**, esta instrucción no trabaja con dos registros fuente, sino con uno solo y un valor numérico fijo.

Tipo de Instrucción: Tipo-I (Inmediato).

Operación: Realiza la suma de un registro con un valor de 16 bits con extensión de signo y con detección de desbordamiento (overflow). El valor inmediato puede ser positivo o negativo.

Sintaxis: `addi rt,rs,inmediato`

rt: Registro destino (donde se guarda el resultado).

rs: Registro fuente.

inmediato: Valor constante (un entero entre -32,768 y 32,767).

EJEMPLO: "Supongamos: $s1 = 50$

`addi s0, s1, -15`

seria: $s0 = s1 + (-15)$ "

3. SLL

En la arquitectura **MIPS32**, la instrucción **sll** (Shift Left Logical o Desplazamiento Lógico a la Izquierda) se utiliza para mover los bits de un registro hacia la izquierda una cantidad específica de posiciones.

Un detalle muy importante es que cada posición desplazada hacia la izquierda equivale a multiplicar el valor por 2. Los espacios vacíos que van quedando a la derecha se rellenan siempre con ceros.

Tipo de Instrucción: Tipo-R (Registro). Aunque usa un valor constante, se codifica en el formato de registro aprovechando un campo especial.

Operación: Desplaza los bits de un registro a la izquierda. Los bits que "salen" por el extremo izquierdo se descartan, y los nuevos espacios a la derecha se llenan con 0.

Sintaxis: `sll rd, rt, shamt`

rd: Registro destino (donde se guarda el resultado).

rt: Registro fuente (el que contiene los bits originales).

shamt: (Shift Amount) Cantidad de posiciones a desplazar (valor constante entre 0 y 31).

EJEMPLO: Se usa mucho para multiplicar por potencias de 2 (2, 4, 8, 16...) de forma muy rápida.

Supongamos: $s1 = 3$ (binario: 0...0011)

`sll s0, s1, 4`

Desplaza 4 bits a la izquierda

Instrucciones de Memoria (Transferencia)

Instrucción	Sintaxis	Tipo	Descripción
<code>lw</code>	<code>lw \$t, offset(\$s)</code>	I	Load Word: Trae un dato de la RAM al registro.
<code>sw</code>	<code>sw \$t, offset(\$s)</code>	I	Store Word: Guarda un registro en la RAM.

4. **LW**

En la arquitectura **MIPS32**, la instrucción `lw` (Load Word o Cargar Palabra) se utiliza para copiar un dato (de 32 bits / 4 bytes) desde la memoria RAM hacia un registro del procesador.

Es una instrucción fundamental porque los procesadores MIPS siguen una arquitectura Load/Store; esto significa que el procesador no puede realizar operaciones aritméticas directamente en la memoria RAM, sino que primero debe "traer" (cargar) los datos a los registros internos para poder trabajar con ellos.

Tipo de Instrucción: Tipo-I (Inmediato).

Operación: Calcula una dirección de memoria sumando un registro base más un desplazamiento constante. Luego, copia el contenido de esa dirección en el registro destino.

Sintaxis: `lw rt, desplazamiento(rs)`

rt: Registro destino (donde se va a guardar el dato traído de la memoria).

desplazamiento: Un valor numérico constante (de 16 bits) que indica cuántos bytes moverse a partir de la dirección base.

rs: Registro base (contiene la dirección de memoria de referencia).

EJEMPLO: Antes de hacer cualquier operación, necesitamos traer el valor de X desde la RAM hacia un registro del procesador.

"Supongamos que el registro `s0` contiene la dirección base de la memoria (0x1000) Y que la variable 'X' está justo en esa dirección inicial."

`lw t0, 0(s0)`

Carga el valor de 'X' desde la RAM al registro `t0`

5. **SW**

En la arquitectura **MIPS32**, la instrucción `sw` (Store Word o Almacenar Palabra) es la operación inversa a `lw`. Se utiliza para copiar un dato de 32 bits (4 bytes) desde un registro del procesador hacia la memoria RAM.

Dado que MIPS es una arquitectura Load/Store, esta es la única instrucción estándar (junto a sus variantes de menor tamaño) capaz de escribir directamente en la memoria RAM para salvar los resultados de nuestros cálculos.

Tipo de Instrucción: Tipo-I (Inmediato).

Operación: Calcula una dirección de destino en la memoria RAM sumando un registro base más un desplazamiento constante. Luego, toma el valor guardado en un registro y lo escribe en esa dirección.

Sintaxis: sw rt, desplazamiento(rs)

rt: Registro fuente (el que contiene el dato que queremos guardar en la RAM).

desplazamiento: Un valor numérico constante (de 16 bits) que indica el desfase en bytes desde la dirección base.

rs: Registro base (contiene la dirección de memoria de referencia).

EJEMPLO: Imagina que ya procesamos el dato (le sumamos 10 usando un addi y el resultado quedó en t1). Ahora queremos guardar ese nuevo valor en la variable Y, la cual se encuentra 4 bytes más adelante en la RAM.

El registro s0 sigue valiendo 0x1000. El registro t1 contiene el nuevo resultado que queremos salvar.

sw t1, 4(s0)

Guarda el valor de t1 en la dirección 0x1004 (Variable Y)

Instrucciones de Comparación y Saltos

Instrucción	Sintaxis	Tipo	Descripción
slt	slt \$d, \$s, \$t	R	Set on Less Than: Si $s < t \rightarrow d = 1$.
slti	slti \$t, \$s, imm	I	Compara registro con constante (inmediato).
beq	beq \$s, \$t, label	I	Salta si $s == t$.
bne	bne \$s, \$t, label	I	Salta si $s \neq t$.
j	j label	J	Salto incondicional.

6. SLT

En la arquitectura MIPS32, la instrucción slt (Set on Less Than o Activar si es Menor Que) se utiliza para realizar comparaciones entre los valores de dos registros.

Es una instrucción clave para controlar el flujo de un programa, ya que permite evaluar condiciones (como los if de los lenguajes de alto nivel) en combinación con instrucciones de salto condicional.

Tipo de Instrucción: Tipo-R (Registro).

Operación: Compara con signo (considerando números positivos y negativos) el valor del primer registro fuente con el segundo. Si el primero es estrictamente menor que el segundo, guarda un 1 en el registro destino; de lo contrario, guarda un 0.

Sintaxis: slt rd, rs, rt

rd: Registro destino (donde se guarda el resultado de la comparación: 0 o 1).

rs: Primer registro fuente (el que se evalúa si es menor).

rt: Segundo registro fuente (contra el que se compara).

7. SLTI

En la arquitectura **MIPS32**, la instrucción slti (Set on Less Than Immediate o Activar si es Menor que el Inmediato) es la variante con constantes de la instrucción slt.

Se utiliza para comparar el valor con signo de un registro contra un valor numérico fijo (un entero inmediato) que se incluye directamente en la propia instrucción de 32 bits.

Tipo de Instrucción: Tipo-I (Inmediato).

Operación: Compara el valor de un registro fuente con un valor inmediato de 16 bits (el cual se extiende con signo a 32 bits antes de realizar la comparación). Si el registro es estrictamente menor que la constante, guarda un 1 en el registro destino; de lo contrario, guarda un 0.

Sintaxis: slti rt, rs, inmediato

rt: Registro destino (guarda el resultado binario: 0 o 1).

rs: Registro fuente (el que se evalúa).

inmediato: Valor constante con signo (entre -32,768 y 32,767).

EJEMPLO:

Supongamos: $s0 = -8$, $s1 = 4$

slt t0, s0, s1

Como -8 es menor que 4, el registro t0 se activa en 1

8. BEQ

En la arquitectura **MIPS32**, la instrucción beq (Branch if Equal o Saltar si son Iguales) es una instrucción de salto condicional. Se utiliza para romper el flujo secuencial del programa y saltar a otra línea de código (etiqueta) únicamente si los valores de dos registros son exactamente iguales.

Es la herramienta nativa que utilizan los compiladores para construir estructuras de control como los if-else, los ciclos while y los ciclos for.

Tipo de Instrucción: Tipo-I (Inmediato).

Operación: Compara el contenido de dos registros. Si son iguales, calcula una nueva dirección de memoria para el Program Counter (PC) sumando un desplazamiento constante. Si no son iguales, el programa continúa normalmente con la siguiente instrucción secuencial.

Sintaxis: beq rs, rt, etiqueta

rs: Primer registro a comparar.

rt: Segundo registro a comparar.

etiqueta: Nombre de la sección de código a donde saltar. A nivel binario, el ensamblador traduce esta etiqueta en un desplazamiento relativo (offset) de 16 bits.

EJEMPLO:

Supongamos: s0 = 75

slti t0, s0, 50

Como 75 NO es menor que 50, t0 se pone en 0

9. BNE

En la arquitectura **MIPS32**, la instrucción bne (Branch if Not Equal o Saltar si NO son Iguales) es la contraparte directa de beq. Es una instrucción de salto condicional que altera el flujo secuencial del programa únicamente si los valores de dos registros son distintos.

Junto con beq, es una de las instrucciones más utilizadas para implementar condicionales if-else y romper o mantener ciclos como el while y el for.

Tipo de Instrucción: Tipo-I (Inmediato).

Operación: Compara el contenido de dos registros. Si los valores no son iguales, calcula una nueva dirección sumando un desplazamiento constante al Program Counter (PC). Si son iguales, la condición falla y se pasa a la siguiente instrucción secuencial.

Sintaxis: bne rs, rt, etiqueta

rs: Primer registro a comparar.

rt: Segundo registro a comparar.

etiqueta: Nombre de la sección de código destino. El ensamblador la convierte internamente en un desplazamiento relativo (offset) de 16 bits.

10. J

En la arquitectura **MIPS32**, la instrucción j (Jump o Salto Incondicional) se utiliza para desviar el flujo del programa de forma obligatoria hacia una dirección específica de la memoria de instrucciones.

A diferencia de beq o bne, no evalúa ninguna condición; el procesador siempre realizará el salto. Además, a diferencia de los saltos condicionales (que son relativos), j utiliza un formato de direccionamiento absoluto directo (con ciertas restricciones del formato del procesador).

Tipo de Instrucción: Tipo-J (Salto / Jump). Es un formato diseñado exclusivamente para maximizar el espacio destinado a la dirección destino.

Operación: Reemplaza los bits del Program Counter (PC) para forzar al procesador a ejecutar la instrucción ubicada en la etiqueta remota.

Sintaxis: j etiqueta

etiqueta: El nombre de la sección de código a donde se quiere ir. El ensamblador calcula la dirección exacta de esa etiqueta y empaqueta sus bits dentro de la instrucción.

EJEMPLO: Un salto forzado e incondicional. No requiere comparar nada, rompe la secuencia de forma directa.

El programa viene ejecutándose secuencialmente

j saltar-bloque (Salta obligatoriamente a la etiqueta)

addi s0, zero, 100 (Esta línea NUNCA se ejecuta)

saltarbloque: addi s0,zero, 5 (El procesador aterriza aquí: s0 = 5)